

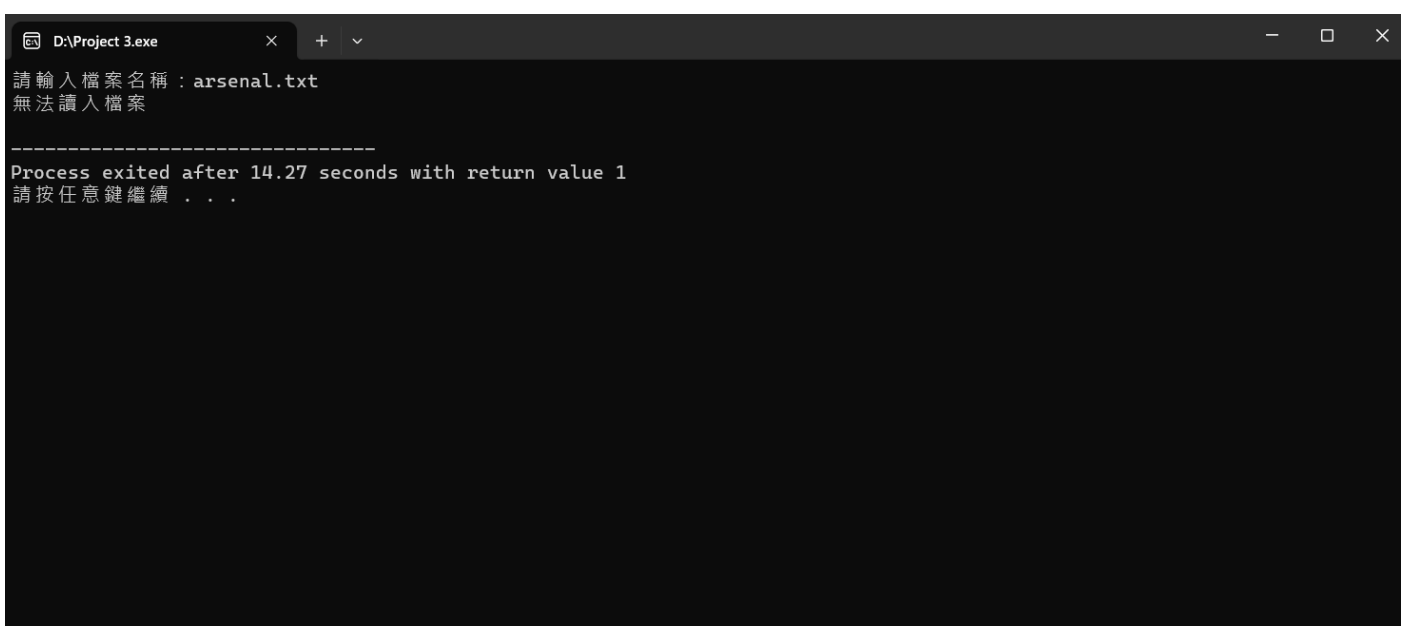
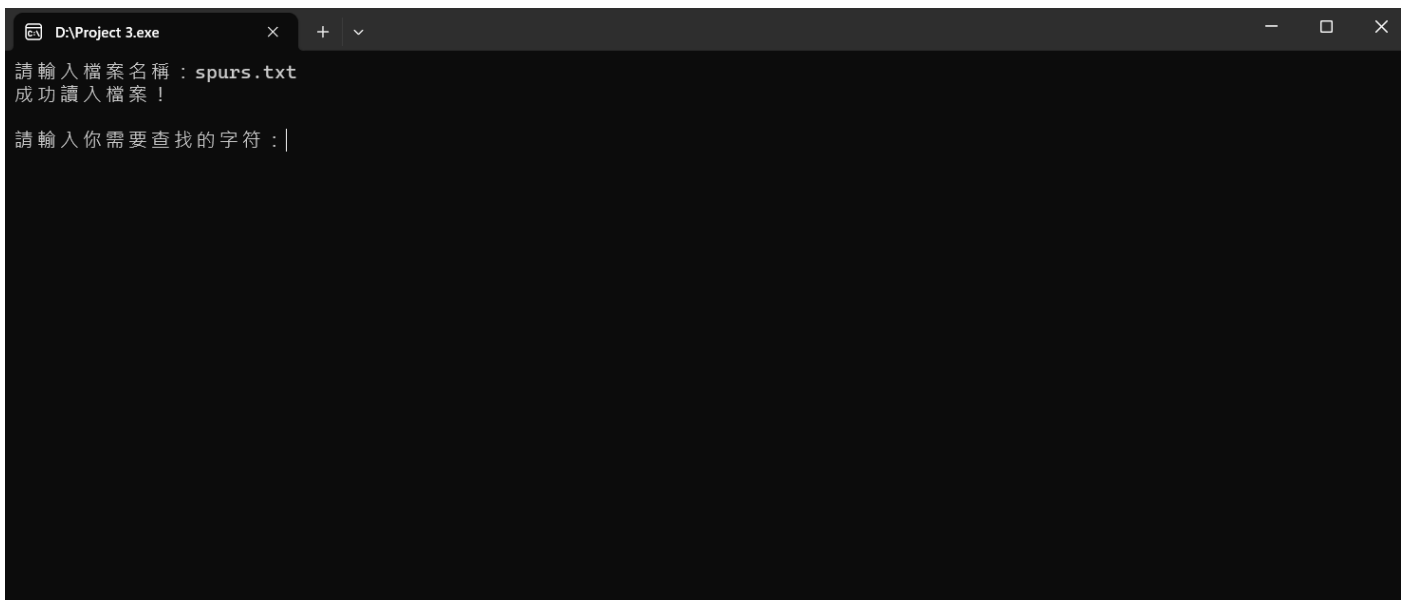
Project 3 实验报告

姓名：陳日康 学号：22336049

1、程序功能简要说明。

程序允许用户在文本文件中查找任何字符或字符串，待统计的词汇集合要求一次输入完毕，即统计工作必须在程序运行一次之后，就全部完成。并提供详细的匹配信息，包括每个匹配项的出现次数和所在行号。

2、程序运行截图，包括计算功能演示、部分实际运行结果展示、命令行或交互式界面效果等。



上图为部分运行结果，首先程序会要求你输入的文档名称，如果文档处在的位置与程序的位置一致，则程序会输出“成功读入档案！”，否则，程序会输出“无法读入档案”。

```
D:\Project 3.exe
請輸入檔案名稱：spurs.txt
成功讀入檔案！

請輸入你需要查找的字符：Tottenham

搜尋結果如下：
行號： 1      出現次數：4
行號： 2      出現次數：2
行號： 4      出現次數：1
行號： 6      出現次數：5
行號： 8      出現次數：2
行號：10      出現次數：2
行號：12      出現次數：1
行號：14      出現次數：1
行號：16      出現次數：2
行號：18      出現次數：4
行號：20      出現次數：1

總出現次數：25

-----
Process exited after 70.71 seconds with return value 0
請按任意鍵繼續 . . . |
```

上图为运行结果，在成功读入文档后，程序会要求用者输入需要查找的字符，然后程序会输出搜寻结果，并且会输出其字符所在的行号以及每行的出现次数。

3、部分关键代码及其说明。

```
9 vector<int> computeLPS(const string& pattern) {
10     int len = 0;
11     vector<int> lps(pattern.size(), 0);
12     int i = 1;
13     while (i < pattern.size()) {
14         if (pattern[i] == pattern[len]) {
15             len++;
16             lps[i] = len;
17             i++;
18         }
19         else {
20             if (len != 0) {
21                 len = lps[len - 1];
22             }
23             else {
24                 lps[i] = 0;
25                 i++;
26             }
27         }
28     }
29     return lps;
30 }
```

`vector<int> computeLPS (const string& pattern)`: 这个函数用于计算给定模式字符串的最长前缀后缀（LPS）数组。LPS 数组用于 KMP 算法的字符串匹配。该函数返回一个整数向量，其中包含每个位置的 LPS 值。

```

36 void kmpSearch(const string& text, const string& pattern, map<int, int>& occurrences, int& totalOccurrences, int lineNumber) {
37     vector<int> lps = computeLPS(pattern);
38     int i = 0;
39     int j = 0;
40
41     while (i < text.size()) {
42         if (isWordCharacter(text[i]) && !isWordCharacter(text[i - 1])) {
43             j = 0;
44         }
45
46         if (pattern[j] == text[i]) {
47             i++;
48             j++;
49         }
50
51         if (j == pattern.size()) {
52             if ((i == text.size() || !isWordCharacter(text[i])) && (i - j == 0 || !isWordCharacter(text[i - j - 1]))) {
53                 occurrences[lineNumber]++;
54                 totalOccurrences++;
55             }
56             j = lps[j - 1];
57         }
58         else if (i < text.size() && pattern[j] != text[i]) {
59             if (j != 0) {
60                 j = lps[j - 1];
61             }
62             else {
63                 i++;
64             }
65         }
66     }
67 }

```

void kmpSearch(const string& text, const string& pattern, map<int, int>& occurrences, int& totalOccurrences, int lineNumber): 这是 KMP 搜索函数，它接受文本字符串、模式字符串、一个记录匹配次数的映射 occurrences，以及一个记录当前行号的整数 lineNumber。它在文本中查找匹配的模式，并记录出现的次数和行号。

```

74     ifstream file(filename);
75     if (!file.is_open()) {
76         cerr << "無法讀入檔案" << endl;
77         return 1;
78     }
79     cout << "成功讀入檔案!" << endl;
80     cout << endl;
81     string pattern;
82     cout << "請輸入你需要查找的字符:";
83     cin.ignore(); // 清除輸入緩衝區
84     getline(cin, pattern);
85     cout << endl;
86
87     map<int, int> occurrences;
88     int totalOccurrences = 0;
89     string line;
90     int lineNumber = 1;
91     while (getline(file, line)) {
92         kmpSearch(line, pattern, occurrences, totalOccurrences, lineNumber);
93         lineNumber++;
94     }
95
96     file.close();

```

ifstream file(filename): 这行代码打开用户指定的文件以供读取。如果文件不存在或打开失败，会输出错误消息。

getline(file, line): 这个函数用于逐行读取文件内容并存储在 line 变量中。

kmpSearch(line, pattern, occurrences, totalOccurrences, lineNumber): 在每一行的文本内容中应用 KMP 搜索，以查找指定模式的匹配项。匹配结果被记录在 occurrences 映射中，包括行号和出现次数。

```

98     cout << "搜尋結果如下：";
99     cout << endl;
100     for (const auto& entry : occurrences) {
101         cout << "行號：" << setw(2) << entry.first << " " << "出現次數：" << entry.second << endl;
102     }
103     cout << endl;
104     cout << "總出現次數：" << totalOccurrences << endl;
105     return 0;
106 }

```

輸出部分：程序使用 cout 輸出匹配結果，包括每個匹配項的行號和出現次數，以及總的匹配次數。

4、程序運行方式簡要說明。

程序開始時，用戶會被提示輸入文件名，程序嘗試打開這個文件以供後續搜索。如果文件無法打開，程序會輸出錯誤消息並退出。如果文件成功打開，用戶將被要求輸入要查找的字符或字符串。程序將逐行讀取文件內容，並對每一行應用 KMP 字符串匹配算法來查找輸入的字符或字符串。如果在一行中找到了匹配項，程序會增加該字符或字符串的出現次數，並記錄該行的行號。程序將所有匹配結果汇总，並在最後輸出每個匹配項的出現次數和所在行號。最後，程序還會輸出總的匹配次數。